

Influencing Game Dynamics in A Roguelike Game Through Procedural Content Generation Using Genetic Algorithm

Josiah Maius S. Ebia, John Carl R. Manalansan, Kate Danielle D. Guino, John Kenneth V. Bunagan

*Computer Science Department, Adamson University
900 San Marcelino Street, Ermita, 1000 Manila*

josiah.maius.ebia@gmail.com
manalansanjohncarl@gmail.com
kdguino@gmail.com
johnkenneth.bunagan0219@gmail.com

Abstract— Procedural content generation and evolving game content based on player preferences, performance, and behavior has been an ever-growing research topic in the field of computer science and game design. In this research, we used genetic algorithms to emphasize the distinct unforgiving nature of *roguelike* games in real time, while the player is in-game, without designing the levels by hand. This will lead to ever changing interactions and inputs from the player and the game environment, having only the game's mechanics and genetic algorithm, besides the player's input, dictate the pace of the game thus influencing the game's dynamics. To demonstrate our proposition, we designed an experimental turn-based 2D roguelike game called *Fireflies*. The core concept of game is to have the probability of the generation of hostile enemies with offensive elements being stronger against that of the player's defense increases as the player progresses through the game.

Index Terms— *Procedural content generation, genetic algorithm, machine learning, game design, roguelike*

I. INTRODUCTION

Procedural content generation (PCG) has been a very popular technique in game design since the past decades. It lets game designers and developers add more replayability to their game. Procedurally generating content will always lead to mostly new and exciting playthroughs every time the player hits the "new game" button. For developers, this design technique saves much more time, in the long run, and resources when creating a game with large amounts of content e.g. The Elder Scrolls: Arena [12].

One of the most popular uses of PCG in game development are in roguelike games. Roguelikes are games

with core mechanics similar to the game Rogue [11], which was released on 1980. Some of the key features of Roguelikes are procedural generation, permadeath, and character progression [6]. Roguelikes, being primarily PCG-based in terms of design, makes it a practical game genre to work on for research titles on PCG and game design theory such as presented in this paper.

Game dynamics are the emergent behavior from the player and game elements when the game's mechanics, which are the particular components of the game such as the game's rules and A.I., are implemented. An example of this is when a player runs from an enemy when her health points are lower than the enemy, or when a player tries to find a key when discovering that a door that is the goal is locked.

In this research, we looked into influencing the emergent behaviors in the game that is game dynamics through procedural generation. This will allow a more personalized play experience for the player, and at the same time, staying true to the unforgiving nature of roguelikes since the game's environment will start to adapt against the player as she progresses.

To accomplish this, we used genetic algorithms, which is a kind of evolutionary algorithm that is inspired by natural selection, to evolve specified elements in the generated content of the game's level depending on the results of the previous level of the game. Lastly, we designed and developed *Fireflies*, an experimental 2D roguelike game, to demonstrate the objectives of this research.

1.1 Significance of the Study

This study can be a model for game designers and developers to improve their understanding on the effectiveness of genetic algorithms in designing roguelike games. It shows a specific way, using genetic algorithms, on

how to make the procedural content generated in roguelike games and the play experience delivered to the player be personalized and more meaningful. This study can also be a framework for students and future researchers in conducting research on game design, procedural content generation, and genetic algorithms.

1.2 Scope and Limitation of the Study

The focus of this research is to utilize genetic algorithms in the procedural generation of content to influence the game dynamics (emergent behavior) in the Fireflies game, giving the player a more personalized experience and at the same time staying true to the nature of roguelikes being both very challenging and fun. Although the methods used in this research project is promising for very challenging games such as that of the roguelike genre, this paper doesn't promise to be as effective on other less challenging games in providing the amount of fun that this can give when applied on roguelikes.

II. REVIEW OF RELATED LITERATURE

2.1 Procedural Content Generation

Procedural content generation (PCG) simply refers to the algorithmic generation of game content with limited or no human contribution [1, 2]. PCG has been used for decades in the games industry. It was primarily used to deliver more content in games that runs in a very limited hardware, popular examples of this are *Rogue* [11] and *The Elder Scrolls: Arena* [12]. In both games mentioned, the developers used PCG to generate game content that would be too large for the existing technology in their time to store and run in memory if predefined. Today, PCG is mainly used to give games more replay value and varied content.

2.2 Roguelike Games

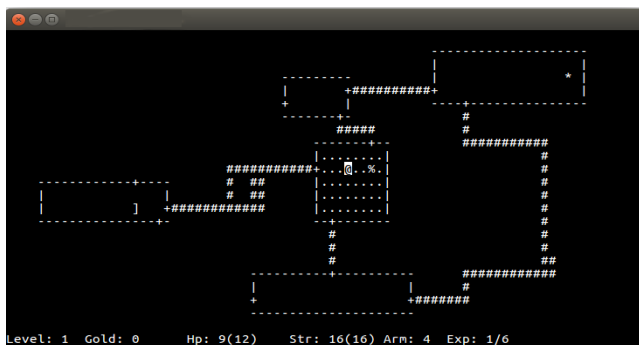


Figure 1. A screenshot of the game *Rogue* running on the Ubuntu Linux terminal

Roguelikes are games whose core features are similar to those of the game *Rogue* [11], thus the name. *Rogue* was developed by Michael Toy and Glenn Wichman which was

initially released on 1980. It was a turn-based dungeon-crawler that used ASCII characters to represent its game content.

The Berlin Interpretation [10] split the factors that defines roguelikes into two, the high value factors and low value factors, the factors listed under the former are the more important features to be found in roguelikes than those under the latter which are likely optional. The following are the high value factors of roguelikes according to The Berlin Interpretation, in no specific order of importance:

- Random environment generation
- *Permadeath* (which stands for permanent death)
- Turn-based
- Grid-based
- Non-modal
- Complexity
- Resource Management
- *Hack'n'Slash*
- Exploration and discovery

Below are the low value factors, in no specific order of importance:

- Single player character
- Monsters are similar to players
- Tactical challenge
- ASCII display
- Dungeons
- Numbers

2.3 Game Dynamics

According to the MDA (*Mechanics-Dynamics-Aesthetics*) framework [7], the consumption of games is broken down into three distinct components which are rules, system, and *fun*, and their design counterpart are mechanics, dynamics, and aesthetics. According to Hunicke et al. [7], mechanic is defined as the particular components of the game, at the level of data representation and algorithms; the dynamics are the run-time behavior of the mechanics acting on player inputs and each other output over time; and aesthetics are the desirable emotional responses evoked in the player, when she interacts with the game system.

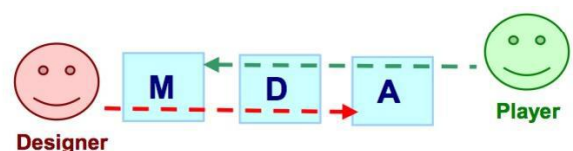


Figure 2. The MDA Framework. Shows the difference in perspective of the player (consumer) and the designer (or developer)

2.4 Genetic Algorithms

Genetic algorithms (GA) are stochastic, parallel search algorithms based on the mechanics of natural selection and evolution [3,5]. It was first introduced by John Holland on 1992 [4] which is very similar to Darwin's theory of natural selection.

A genetic algorithm starts off with a population of elements, or *chromosomes*, which carry their own genetic information commonly referred to as genes. The algorithm will then evaluate the *fitness* of each of the chromosomes in the population, the evaluation function of a genetic algorithm depends on the problem being solved. A chromosome's *fitness value* will determine its likelihood of being picked as a parent for the new generation of likely more optimal chromosomes. When the parent chromosomes are picked the algorithm will perform *crossover* and *mutation*. The crossover function will combine genes from the parent chromosomes to form a new chromosome for the next generation, a mutation may occur once the crossover is done which will mutate a certain gene that is unique from its parents. The above explanation may also be described as:

$$P = \{x | x \text{ is the return value of } f(c_1, c_2, \dots, c_n)\}$$

where f is a genetic function that takes n number of chromosome c and returns a child chromosome x , and f runs N times where N is the required number of chromosomes for the population set P in the next generation. The genetic algorithm stops when it produces the most optimal result that satisfies the problem.

III. PROPOSED METHODOLOGY

3.1 The Fireflies Game

Fireflies is an experimental 2D turn-based roguelike game. In *Fireflies*, the player is thrown in a series of procedurally generated levels with harmful and helpful game elements such as enemies and loot. The main objective of the player is to utilize her resources and strategize to find a way to finish the level's objectives and reach the goal exit of each level.

Fireflies features procedurally generated levels, this means that certain parameters (e.g. the x and y coordinates) of every game object in every new level that the player enters isn't predefined and will not be always the same. Each dungeon level generated will contain matching rooms and corridors, enemies, loot, and a goal.

The game is heavily turn-based, all actors in the game including the player have one action per turn. Every turn, an actor either attacks or moves to a tile to change its position, as shown in Figure 3. The turn of every other actor in the game starts after the player finishes her turn.

During the player's turn, she can choose to attack instead of moving to another tile. This will cause the player to enter the attack phase, where a marker will appear one (1) tile from her position and she may choose to attack one tile north, east, south, or west, this is shown in Figure 4. The player may also cancel the attack and return to her normal state, doing this will not cost the player a turn and she may either choose to move or attack again. The core combat mechanics in the game is adapted from the mechanics of the traditional *rock, paper, and scissors* (RPS) game where 2 players

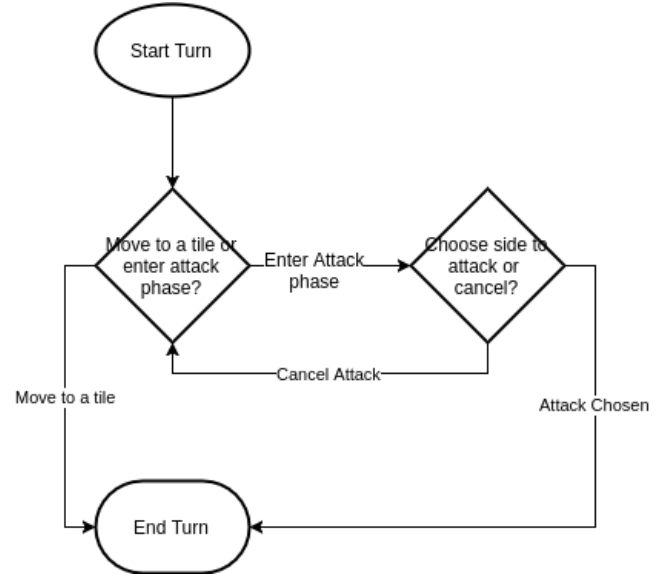
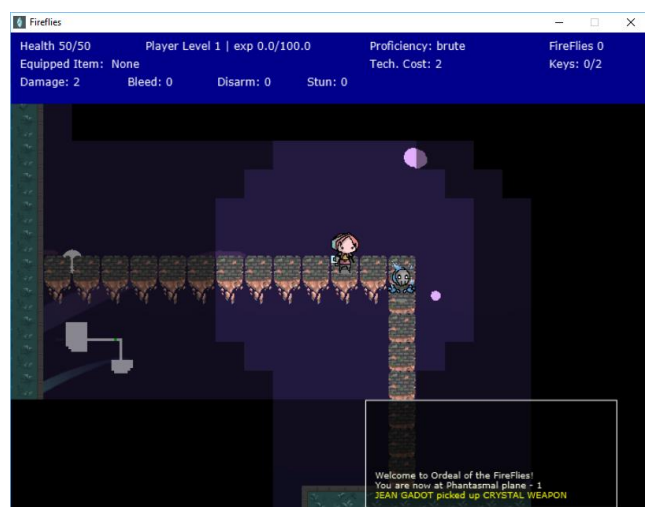


Figure 3. The general flow of an actor's turn in the game

choose from the three (rock, paper, and scissors) per turn and the victor are determined by the very simple rule: rock beats scissors, scissors beats paper, and paper beats rock. We chose to adapt the concept for the combat system of our game because it proves to be one of the most balanced mechanics in a game available, since all the players are given a similar set of tools and rules so that everyone has the same chances of winning given that they know how to play and use their tools correctly. In *Fireflies*, the RPS system is implemented in the attacks and defense of both the player and enemy actors, where one attack is stronger than the defense of the other. The rock, paper, and scissors are represented as *brute*, *stable*, and *cut techniques*, respectively. The player may choose to attack using any of the three techniques, but the enemies will only have one technique for offense and defense. Each technique has a 10% chance to produce a unique effect on hit either on the player or enemy characters. The brute technique unique effect is *stun* which makes a character unable to do anything for a number of turns. Stable technique has a *disarm* effect which

The player may be *proficient* with a technique (*brute, stable, cut*). She may choose to become proficient with only one technique at a time. This will make her take less damage when an enemy with a technique that is weaker against the player attacks her, but this will also increase the damage of an enemy with a technique that is stronger than her proficiency. Switching her proficient type requires spending two (2) *firefly essences* which can be collected after defeating an enemy. Each enemy drop one (1) *firefly essence* when their health reaches zero (0).



Fireflies

Health 28/60 Player Level 2 | exp 160.0/400.0 Proficiency: brute Fireflies 9

Equipped Item: Crystal Weapon Tech. Cost: 2 Keys: 1/1

Damage: 3 Bleed: 0 Disarm: 0 Stun: 0

JEAN GADOT dealt 3 damage to WRAITH
 WRAITH dealt 4 damage to JEAN GADOT
 JEAN GADOT dealt 4 damage to WRAITH
 WRAITH died. You gained 40.0 exp!
 JEAN GADOT dealt 6 damage to WRAITH
 WRAITH[stable] died. You gained 40.0 exp!

3.2 The Game Controls

Pressing the spacebar will make the player enter *attack mode* and will show an indicator and will allow the player to attack. Once the indicator is shown, the player can choose which direction she will attack using the arrow keys. When the player determines which direction she will attack, the player can choose which technique she will use in order to defeat an enemy using the Q, W and E keys. The Q key stands for the *brute* technique, the W key stands for *stable*, and the E key stands for *cut*.

Keys J, K, and L corresponds to changing proficiencies to brute, stable, and cut, respectively. Pressing I will open the inventory menu where the player may navigate through her inventory of items using the arrow keys and then use or delete an item using the Enter and the delete keys, respectively.



Figure 4.3 A screenshot of *Fireflies* showing the inventory menu.

3.3 The Genetic Algorithm

The game's genetic algorithm (GA) resides in the level generator, where the enemies are also generated. The GA takes in the population of enemy *non-playable character* (NPC) and evaluates the fitness of every NPC in the population. The NPCs' fitness value is based on their remaining *health* after the player has exited the level in which they are in. This enables the enemies of the previous level that the player did not or cannot kill or find to get significantly better chances of being a *parent* than the other enemy NPCs. The method of calculating the fitness of the

enemies is a form of *artificial evolution* similar to Karl Sims' *Genetic Images* [9], where the enemies' fitness is determined by the user (in this case, the player) and not absolutely by a sole fitness function.

When picking an enemy as a parent, we used the accept-reject method where a value with a maximum equal to the sum of all remaining health from the enemy and a minimum of zero (0) is randomized. The randomized value will be subtracted by each of the enemies in the population until the random value reaches zero, then the last enemy observed will be picked as a parent. This process is repeated for the partner parent.

After the parents, which are the enemy entities, are picked, they will undergo the crossover function. In the crossover function, the *technique* element of the offspring will be randomly picked from the two parents, the same is also done for the enemy type which governs how the enemy looks. The offspring will then undergo the mutation function where there will be a chance that the offspring may change appearance or traits that are different from the parents'.

3.4 Libraries and Tools Used

Java

The game was written from scratch using the Java programming language, which is an object-oriented language. The JavaFX library was used to handle the graphics of the game.

3.5 Evaluation Metrics and Procedures

To evaluate the game *Fireflies*, a series of test plays is conducted. The player will be playing up to, at most, the 10th level of the game. Results of each playthrough will then be stored and graphed to see how the genetic algorithm performed.

IV. RESULTS AND DISCUSSION

4.1 Results

The following are 8 of the tabular representation of data from the multiple monitored test runs of the game. We only picked 8 of the data set since most of them has similar trends in the data. The data monitored per level per game are the number of enemies killed from the previous level, the number of enemies spawned in the current level and the proficiency changes of the player:

Game 1	Killed	Spawned	Killed	Spawned	Killed	Spawned
Level	0	1	1	2	2	3
Brute	0	5	2	2	2	0
Stable	0	5	1	11	6	14
Cut	0	3	3	1	1	2
Proficiency	Brute		Brute > Cut		Cut	DIED

Figure 5.1. Results of game 1

GAME 2	KILLED	SPAWNED	KILLED	SPAWNED
Level	0	1	1	2
Brute	0	2	2	1
Stable	0	3	3	10
Cut	0	3	1	15
Proficiency	brute	> cut	cut > brute	DIED

Figure 5.2. Results of game 2

Game 3	Killed	Spawned	Killed	Spawned	Killed	Spawned
Level	0	1	1	2	2	3
Brute	0	5	3	2	1	16
Stable	0	6	0	8	6	2
Cut	0	3	2	5	2	2
Proficiency	Brute		Brute > Cut > Stable		Stable	DIED

Figure 5.3. Results of game 3

Game 4	Killed	Spawned	Killed	Spawned	Killed	Spawned
Level	0	1	1	2	2	3
Brute	0	13	5	6	4	12
Stable	0	5	0	4	1	7
Cut	0	6	3	2	1	3
Proficiency	Brute > Stable		Stable > Brute		Brute > Stable > Cut	DIED

Figure 5.4. Results of game 4

Game 5	Killed	Spawned	Killed	Spawned
Level	0	1	1	2
Brute	0	3	0	5
Stable	0	4	1	9
Cut	0	5	4	4
Proficiency	Brute		Brute > Cut	

Game 5	Killed	Spawned	Killed	Spawned
Level	2	3	3	4
Brute	0	2	0	3
Stable	2	10	8	7
Cut	0	0	0	0
Proficiency	Cut		Cut	

Game 5	Killed	Spawned	Killed	Spawned
Level	4	5	5	6
Brute	0	10	4	12
Stable	6	2	0	0
Cut	0	0	0	6
Proficiency	Cut > Stable		Stable	DIED

Figure 5.5. Results of game 5

Game 6	Killed	Spawned	Killed	Spawned
Level	0	1	1	2
Brute	0	4	1	2
Stable	0	4	2	6
Cut	0	4	0	4
Proficiency	Brute		Brute	DIED

Figure 5.6. Results of game 6

Game 7	Killed	Spawned	Killed	Spawned	Killed	Spawned
Level	0	1	1	2	2	3
Brute	0	0	0	0	0	1
Stable	0	4	0	2	2	10
Cut	0	6	1	3	3	5
Proficiency	Brute		Brute		Brute	DIED

Figure 5.7. Results of game 7

Game 8	Killed	Spawned	Killed	Spawned
Level	0	1	1	2
Brute	0	0	0	1
Stable	0	5	3	9
Cut	0	1	0	6
Proficiency	Brute		Brute	
Game 8	Killed	Spawned	Killed	Spawned
Level	2	3	3	4
Brute	1	1	0	2
Stable	4	7	1	3
Cut	5	4	2	11
Proficiency	Brute		Brute	DIED

Figure 5.8. Results of game 8

From the results shown above, there is a clear trend in the enemy generation of a certain type where the lesser the number of enemies spawned previous level minus the number of enemies killed in the previous level, means a higher chance of the enemy of that certain type to be generated. Take the test results from *Game #7* for example, where the number of stable type enemies generated are increased since the gap between the numbers of stable type enemies killed versus spawned is high, they were hunted less than they are generated compared to the other element types. Often times, the player will change her proficiency depending on the types of enemies generated. An example is the *Game #4* where on the third level, the player changed her proficiency from brute to stable since the number of brute type enemies generated outnumbers the stable and cut type enemies.

However, there is also a problem shown in some of the results. Since the genetic algorithm observes only one significant variable as basis for the enemies' fitness, which is the enemy's remaining health, the parent picking becomes sort of binary in value. When the player encounters an enemy, the most rational decision is for her to either kill it or avoid it, so most of the time the fitness value of each of the enemies when they undergo the genetic algorithm is as good as either 0 or 1. As you see, there are results such as in *Game #5* where the brute type enemies have completely dominated the generation algorithm since no one else there is enough to compete, this will lead to very uninteresting playthroughs with little to no variety, especially for a roguelike game.

Another flaw in the project, as seen in the results above, is the number of levels the player reaches in the game. From the total eight (8) game results shown, the player only reached up to three (3) levels in four (4) of them, and up to two (2) levels in two of the games. This is due to players finding the game very challenging specially at the start. The cause of this is the lack of proper mapping of key game elements such as the spawn location of the enemies and loot. Most of the game elements are simply scattered randomly across the generated level, the generator does not take into account how the game elements are placed.

Having a better overall game design, observing more variables for the genetic algorithm, and guiding the generator with something like a grammar system would have avoided the problems mentioned above.

V. CONCLUSIONS AND RECOMMENDATIONS

5.1 Conclusion

Using genetic algorithms to create a play dynamic that is always changing based on the actions of the player, in this case her interaction with enemies in combat, in a roguelike game is very feasible. Since the roguelike genre is built on being both fun and challenging and genetic algorithms are good for optimization, the game genre and machine learning concept complements one another. In *Fireflies*, although the implementation was not perfect, the results have shown promise regarding the concept of implementing genetic algorithms in roguelike games. It shows that the genetic algorithm can make roguelike elements as challenging as a human designer would. The main flaw of *Fireflies* is the game's mechanics and design overall has hindered the algorithm's ability to yield better results. But, given the time we have to develop the project, we can fairly say that the project is successful. Since *Fireflies* is just an experimental game, it still requires a lot of play-testing and adjustments, as well as its level generator, to show far better results in the future.

5.2 Recommendations

The methods discussed above has shown much promise in using genetic algorithms in games, especially in games under the roguelike genre. We, the researchers, recommend taking the research a step further by implementing a grammar system to guide the generation process of the game elements. A grammar system will be a great tool for achieving a better and more human design-like procedurally generated game level structures.

We would also recommend the future researchers to give more game variables for the game's algorithm to observe. An example of this is observing the player's movement patterns, how the player moves when an enemy reaches her line of sight, and others. A better planned and studied game design would also make a much better platform for the genetic algorithm to work on.

ACKNOWLEDGEMENT

We would like to thank our family for their never-ending support in the making of this project and for constantly reminding us to not skip our meals and take

enough rest. A special thanks to our friends, for testing our game and suggesting elements that would make the game more *fun*, and to JM Ebia, our team lead, who never gave up on his teammates and led us to the betterment of the field we have chosen.

We would also like to express our gratitude to Ma'am Mae Blanco, Sir Edward Bustillos and Sir Billie De Leon for their guidance and suggestions for the greater good of this project. And to you, players who will play the game.

REFERENCES

- [1] Julian Togelius, Georgios N. Yannakakis, Kenneth O. Stanley, and Cameron Browne. Search-Based Procedural Content Generation: A Taxonomy and Survey. *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 3, 2011.
- [2] Julian Togelius, Alex J. Champandard, Pier Luca Lanzi, Michael Mateas, Ana Paiva, Mike Preuss, and Kenneth O. Stanley. Procedural Content Generation: Goals, Challenges and Actionable Steps. *Dagstuhl Follow-Ups* 6, 2013.
- [3] Nicholas Cole, Sushil J. Louis, and Chris Miles. Using a genetic algorithm to tune first-person shooter bots. *Evolutionary Computation, 2004. CEC2004. Congress on. Vol. 1. IEEE.*, 2004.
- [4] John Holland. Genetic algorithms. 1992.
- [5] Darrell Whitley. A genetic algorithm tutorial. *Statistics and computing* 4.2 (1994): 65-85, 1994.
- [6] Simon Almgren, Leo Anttila, David Oskarsson, Fabian Sorensen, and Samuel Tiensuu. *Astrogue: A Roguelike*. 2014.
- [7] Robin Hunicke, Marc LeBlanc, and Robert Zubek. MDA: A formal approach to game design and game research. *Proceedings of the AAAI Workshop on Challenges in Game AI. Vol. 4. No. 1. 2004*, 2004.
- [8] Gillian Smith, Elaine Gan, Alexei Othenin-Girard, and Jim Whitehead. PCG-based game design: enabling new play experiences through procedural content generation. *Proceedings of the 2nd international workshop on procedural content generation in games. ACM, 2011*, 2011.
- [9] Karl Sims. Genetic Images. <http://www.karlsims.com/genetic-images.html>, 1993.
- [10] The Berlin Interpretation. <http://www.roguebasin.com>, 2008.
- [11] Michael Toy, Glenn Wichman. *Rogue. (Game)* 1980.
- [12] Bethesda Softworks. *The Elder Scrolls: Arena. (Game)* 1994.